

Doc. no.:	P3139R0
Date:	2024-5-20
Audience:	LEWG
Reply-to:	Zhihao Yuan <zhihao.yuan@broadcom.com> Jordan Saxonberg <jordan.saxonberg@broadcom.com>

Pointer cast for `unique_ptr`

- Pointer cast for `unique_ptr`
 - Introduction
 - Prior Art
 - Motivation
 - Design
 - Technical Specification
 - Implementation Experience
 - Acknowledgements

Introduction

We propose `unique_ptr` overloads for `std::const_pointer_cast` and `std::dynamic_pointer_cast`. For each kind of cast, we allow users to choose between either using the defaulted deleter or preserving the original deleter type for each kind of cast.

The table following illustrates the simpler case of the two where users expect defaulted deleters in the resulting types.

Given an API:

```
auto GetClient() -> std::unique_ptr<const Client>;
```

C++23

⚠ Owing raw pointer

```
std::unique_ptr<Client> client;  
client.reset(const_cast<Client*>(GetClient().release()));
```

✗ Leak resources if `dynamic_cast` fails

```
void UseV2Client(std::unique_ptr<Client>&& client)  
{  
    std::unique_ptr<ClientV2> v2;  
    // ...  
    v2.reset(dynamic_cast<ClientV2*>(client.release()));  
}
```

P3139

✓

```
std::unique_ptr<Client> client;  
client = const_pointer_cast<Client>(GetClient());
```

✓

```
void UseV2Client(std::unique_ptr<Client>&& client)  
{  
    std::unique_ptr<ClientV2> v2;  
    // ...  
    v2 = dynamic_pointer_cast<ClientV2>(std::move(client));  
}
```

Prior Art

Boost.SmartPtr (https://www.boost.org/doc/libs/1_63_0/boost/pointer_cast.hpp) ships all four casts (`dynamic_pointer_cast` , `static_pointer_cast` , `const_pointer_cast` , and `reinterpret_pointer_cast`) that create `std::unique_ptr<T>` by releasing `std::unique_ptr<U>` since 2016.

Motivation

Improve resource safety

The need for pointer casts between `unique_ptr`s was spotted in code reviews from independent parties. Without coincidence, the conclusions were to use `.release()`, a resource-unsafe API, as a one-off solution. Such a practice encourages the use of unsafe constructs and potentially breaks future code as a result. The standard C++ should encourage the opposite.

Express the intent of pointer cast via established vocabularies

When people look for pointer casts between smart pointers, they look for `std::dynamic_pointer_cast`, `std::const_pointer_cast`, etc. These names resemble `dynamic_cast` and `const_cast` and work for `std::shared_ptr` already. There is no simpler way to express the same intent and no reason to find a different set of names.

Standardize existing practices

The `cpplang` (<https://cppalliance.org/slack/>) Slack workspace rediscovers `dynamic_pointer_cast(std::unique_ptr<T>&&)` on a yearly basis, albeit `boost::dynamic_pointer_cast(std::unique_ptr<T>&&)` existed before the group's birth. Some of the work supports preserving the incoming deleter type. It's time to consider adopting the working parts from Boost and explore the recurring extension.

Design

`unique_ptr` differs from `shared_ptr` in a few significant ways. Obviously, we can only cast from an rvalue of `unique_ptr` by moving its ownership. The other differences that made impacts on the design are:

1. A `shared_ptr<T>` carries a type-erased deleter, while `unique_ptr<T, D>`'s deleter is a part of the type. The seemingly intuitive `unique_ptr<T>` to `unique_ptr<U>` actually requires replacing `std::default_delete<T>` with `std::default_delete<U>`, which may not apply to the customized deleters.
2. A casted `shared_ptr<U>` is only an alias to the original `shared_ptr<T>`. The newly created object requires no deleter, and you can expect the original deleter to be able to delete the uncasted pointer. Meanwhile, `unique_ptr<U, D>`, in general, must deal with the question "whether `D` can delete the casted pointer."
3. A `shared_ptr<T>` owns a "real" pointer, while `unique_ptr<T, D>` can customize its pointer type as indicated by its pointer typedef. Converting between the pointer types may create loopholes as the `unique_ptr<T[], D>` specializations reused this mechanism.

It turns out that using `unique_ptr` with a type-erased deleter is not uncommon in the industry. It does not have to be as sophisticated as something that calls into `memory_resource`. A deleter that takes a pointer to a polymorphic base class is possibly a type-erased deleter. Even `&std::free` is a legitimate type-erased deleter. So when designing the APIs to cast between `unique_ptr`s, we would like to support both expectations: **one set of APIs to cast `unique_ptr<T>` to `unique_ptr<U>` and the other set to cast `unique_ptr<T, D>` to `unique_ptr<U, D>`**. In other words, one defaults the deleter, and the other one preserves the deleter.

However, the set of casts we can safely perform in practice is not without boundaries with both API styles. According to our preliminary survey using GitHub code search, `static_cast`s between `unique_ptr`s using the `.release()` trick almost always perform downcast in the hope of gaining performance over `dynamic_cast` by sacrificing safety, and `reinterpret_cast`s between `unique_ptr`s only retrieve byte sequences. The authors consider both use cases require expertise and cannot be a part of the intuitive APIs, which are supposed to be safe by default. On the other hand, type-erased APIs for these types of casts would not only be expert-only but also serve no use case, as we know so far. Therefore, this paper proposes **only `dynamic_pointer_cast` and `const_pointer_cast` between `unique_ptr`s**.

Attention to safety is also reflected in the proposed APIs. For example, in the API to downcast `unique_ptr<T>` to `unique_ptr<U>`, we require `U` to have a virtual destructor. This is not required in the deleter-preserving API since the behavior of the deleter is unknown. But when the deleter is known to be `default_delete<U>`, we can prevent undefined behavior ahead of time. In some cases, prior knowledge of the default deleter reduces the amount of checks. For example, `unique_ptr<T[], D>::pointer` may also be `T*`, which requires extra checks to prevent accidentally creating `unique_ptr` that manages `new[]`-ed resources with a non-array deleter and such. The following chart summarizes the **guardrails in the proposed APIs** beyond the underlying calls to the `unique_ptr` constructors.

	<code>unique_ptr<T> → unique_ptr<U></code>	<code>unique_ptr<T,D> → unique_ptr<U,D></code>
<code>const_pointer_cast</code>	Valid to <code>const_cast</code> from <code>T*</code> to <code>U*</code>	Valid to <code>const_cast</code> from <code>unique_ptr<T, D>::pointer</code> to <code>unique_ptr<U, D>::pointer</code> ; Either <code>T</code> and <code>U</code> both are array types, or neither
<code>dynamic_pointer_cast</code>	Valid to <code>dynamic_cast</code> from <code>T*</code> to <code>U*</code> ; <code>U</code> has a virtual destructor	Valid to <code>dynamic_cast</code> from <code>unique_ptr<T, D>::pointer</code> to <code>unique_ptr<U, D>::pointer</code> ; Neither <code>T</code> nor <code>U</code> is an array type

Technical Specification

```
template<class T, class U>
constexpr unique_ptr<T> dynamic_pointer_cast(unique_ptr<U>&& r) noexcept;
```

Constraints: `dynamic_cast<T*>((U*)nullptr)` is a valid expression.

Mandates: `has_virtual_destructor_v<T>` is true.

Preconditions: The expression `dynamic_cast<T*>(r.get())` has well-defined behavior.

Effects: Equivalent to:

```
if (auto p = dynamic_cast<T*>(r.get()))
    return (void)r.release(), unique_ptr<T>(p);
else
    return nullptr;
```

[Note 1: The seemingly equivalent expression `unique_ptr<T>(dynamic_cast<T*>(r.get()))` can result in undefined behavior, attempting to delete the same object twice. —end note]

```
template<class T, class D, class U>
constexpr unique_ptr<T, D> dynamic_pointer_cast(unique_ptr<U, D>&& r) noexcept;
```

Constraints: `dynamic_cast<unique_ptr<T, D>::pointer>(declval<typename unique_ptr<U, D>::pointer>())` is a valid expression.

Mandates: Neither `T` nor `U` is an array type.

Preconditions: The expression `dynamic_cast<unique_ptr<T, D>::pointer>(r.get())` has well-defined behavior.

Effects: Equivalent to:

```
if (auto p = dynamic_cast<unique_ptr<T, D>::pointer>(r.get()))
    return (void)r.release(), unique_ptr<T, D>(p, std::forward<D>(r.get_deleter()));
else if constexpr (!is_pointer_v<D> && is_default_constructible_v<D>)
    return nullptr;
else if constexpr (is_copy_constructible_v<D>)
    return unique_ptr<T, D>(nullptr, r.get_deleter());
```

```
template<class T, class U>
constexpr unique_ptr<T> const_pointer_cast(unique_ptr<U>&& r) noexcept;
```

Constraints: `const_cast<T*>((U*)nullptr)` is a valid expression.

Effects: Equivalent to: `return unique_ptr<T>(const_cast<T*>(r.release()));`

[Note 2: The seemingly equivalent expression `unique_ptr<T>(const_cast<T*>(r.get()))` can result in undefined behavior, attempting to delete the same object twice. —end note]

```
template<class T, class D, class U>
constexpr unique_ptr<T, D> const_pointer_cast(unique_ptr<U, D>&& r) noexcept;
```

Constraints: `const_cast<unique_ptr<T, D>::pointer>(declval<typename unique_ptr<U, D>::pointer>())` is a valid expression.

Mandates: `is_array_v<T> == is_array_v<U>` is true.

Effects: Equivalent to: `return unique_ptr<T, D>(const_cast<unique_ptr<T, D>::pointer>(r.release()), std::forward<D>(r.get_deleter()));`

Implementation Experience

Compiler Explorer

Here is a full implementation: [n3Kh376hs](https://godbolt.org/z/n3Kh376hs) (<https://godbolt.org/z/n3Kh376hs>)

The snippet below implements the variant of `dynamic_pointer_cast` that preserves the deleter type (i.e., supports type-erased deleter).

```
template<class T, class D, class U>
    requires(requires(unique_ptr<U, D>::pointer p) {
        dynamic_cast<unique_ptr<T, D>::pointer>(p);
    })
constexpr auto dynamic_pointer_cast(unique_ptr<U, D>&& r) noexcept
    -> unique_ptr<T, D>
{
    static_assert(!is_array_v<T> && !is_array_v<U>,
        "don't work with array of polymorphic objects");
    if (auto p = dynamic_cast<unique_ptr<T, D>::pointer>(r.get()))
    {
        r.release();
        return unique_ptr<T, D>(p, std::forward<D>(r.get_deleter()));
    }
    else if constexpr (!is_pointer_v<D> && is_default_constructible_v<D>)
    {
        return {};
    }
    else if constexpr (is_copy_constructible_v<D>)
    {
        return unique_ptr<T, D>(nullptr, r.get_deleter());
    }
    else
    {
        static_assert(false, "unable to create an empty unique_ptr");
    }
}
```

Acknowledgements

Thank Broadcom Software for supporting the work.